

网络问题汇总---排查项

2026年3月2日 23:28

tips

1. 搜索k8s官网文档，搜索cce官网文档，搜索友商云文档
2. 搜索wiki:
3. 搜索bug单:
4. 搜索需求单:
5. 搜索GitHub issue:
6. 如果cpu内存io负载网络等使用率没有规律可以尝试热迁移:
7. ifconfig网卡正常吗？重启网卡试下？
8. pod使用hostnetwork模式试下:
9. 抓包，必要时全量抓包！:
10. 注意把service后端的每个pod都curl下:
11. 节点上是否有定时任务
12. 和正常节点对比下内核版本，ipvlan版本，ovs版本
13. 和正常pod对比下os版本，路由，内核参数
14. 问下用户有没有做过什么变更，升级过内核版本，修改过内核模块，改过路由，改过iptables，升级过组件，安装了什么软件，改过内核参数？
15. 重启组件，重启节点，重置节点，重买集群。重置节点也不行，用户做过什么操作？切换过vpc？
16. 根据用户的配置，复现
17. pod单独部署到一个节点，不要和其他pod混部
18. 偶现的，持续数秒的，第一时间去看虚拟机是否有迁移
19. 访问慢：重点在：域名解析慢、是否有走公网、nat eip带宽使用率、跨节点路由、cpu高，磁盘io高也会影响网络、物理机虚拟化、抓包是否有重传、内核参数不合理、业务代码，业务参数

一、pod

1、查看pod监控

cpu：业务逻辑

内存：gc

load：线程数

磁盘io

pps：

2、pod cpu限流

limit是否合理

二、节点

监控：

1、cpu

- 2、内存
- 3、load
- 4、磁盘io
- 5、出入pps bps
- 6、网络连接数：1) 内核日志：ddos攻击导致连接数打满 2) 热迁移导致网络短暂中断
- 7、虚拟机新建连接数
- 8、netstat 连接数过多或泄漏
- 9、iptables规则过多导致内核处理延迟高

iptables

iptables -nvL -t {表名, 默认是filter表}

iptables-save |grep -i reject

iptables-save |grep -i drop

iptables-save |grep -i return

iptables-save |grep -i masquerade

iptables-save |grep -i nat

iptables-save |grep -i redirect

history |grep iptables

connect: no route to host一般是iptables reject原因

节点路由

和正常节点路由对比下, 常见的可能性有:

- 1、由于某种原因, 用户手动修改过节点路由;
 - 2、节点启动了某种网络相关的进程、安装了网络相关的pod、插件等导致路由被自动修改
- dmesg内核日志/message日志

dmesg -T |grep "dropping packet"

dmesg -T |grep "possible syn flooding"

dmesg -T |grep "time wait bucket table overflow"

调大net.ipv4.tcp_max_tw_buckets

dmesg -T |grep "nf_conntrack table full"

最大值sysctl -a |grep net.nf_conntrack_max = 262144

查看当前连接数 cat /proc/net/nf_conntrack |wc -l

连接跟踪表tcp连接的老化时间

net.netfilter.nf_conntrack_tcp_timeout_established = 432000

调整上限sysctl -w net.netfilter.nf_conntrack_max = 2097152

方案:

- A. 调整跟踪表门限 ---- 持续反复仍可能达到限制值
- B. 重启iptables -- 可能影响业务 造成中断
- C. 内核协议栈关闭已知的连接或等待其它会话超时 -- 需将nf_conntrack重构实现
- D. 改小老化时间 - 在极端场景下, 仍有可能出现连接表满

E. fdsa增加对nf_conntrack值的监控，当发现其接近门限或触发自愈合与产品SE/TSE 反复讨论评估，认为方案E为优选。

后因可维护性SE不同意自愈复位，经CCB讨论，方案修改为：

通过FDSA监控系统nf_conntrack数量，当其超过max值50%之后，则修改tcp长连接的老化时间netfilter.nf_conntrack_tcp_timeout_established为10分钟；后面如发现其小于门限值20%，则重新恢复为默认值(432000)。

内核参数

k8s必须开启net.ipv4.ip_forward = 1

节点有大量CLOSE_WAIT链接未释放，占用内存，容器访问服务慢

```
sysctl -a | grep -E
```

```
"net.ipv4.tcp_tw_recycle|net.ipv4.tcp_tw_reuse|net.ipv4.tcp_timestamps|net.ipv4.tcp_max_syn_backlog|net.core.somaxconn"
```

客户端为snat环境+服务端开启timestamp+recycle：

tcp_tw_recycle参数和tcp_timestamps参数一起开启的时候会触发内核TCP/IP协议栈的per-host的PAWS机制，60s内同一个源客户端的连接请求中的时间戳必须是递增的，否则数据包将会被丢弃

```
sysctl -a | grep net.ipv4.tcp_tw_recycle
```

```
sysctl -a | grep net.ipv4.tcp_tw_reuse
```

检查 PAWS 丢包

```
$ cat /proc/net/netstat | grep TcpE | awk '{print $15, $21}'
```

PAWSPassive ListenDrops

96305 96305

```
sysctl -a | grep net.ipv4.tcp_timestamps //问公网超时connect timed out,需关闭。
```

```
netstat -s | grep reject
```

```
netstat -s | grep "time stamp"
```

检查半连接队列超时：

```
sysctl -a | grep net.ipv4.tcp_max_syn_backlog cat /proc/sys/net/ipv4/tcp_max_syn_backlog
```

```
netstat -s | grep "SYNs to LISTEN"
```

net.ipv4.tcp_max_syn_backlog, 默认值受内存大小的影响，与机器内存成比例增加。最小为128，默认值最大为4096。

检查全连接队列超时，容器默认somaxconn是128，业务量大的话容器出现超时

io.netty.channel.ConnectTimeoutException: connection timed out

```
sysctl -a | grep net.core.somaxconn
```

```
cat /proc/sys/net/core/somaxconn
```

```
netstat -s | grep overflowed
```

执行ss -lnt

对于 LISTEN 状态，Send-Q 表示 accept queue 的最大限制大小，Recv-Q 表示其实际大小。

如果netstat -s命令不可用，可以执行cat /proc/net/netstat | awk '/TcpExt/{ print \$20,\$21 }'查看

客户端并发量大，源端口耗尽

不同的os，不同的命令行工具，不同的客户端，报错不一致；报错一般为Cannot assign requested address 或Address not available或bind: Address in use

通过对比ESTABLISHED状态的连接数和net.ipv4.ip_local_port_range确认

```
netstat -an |grep ESTABLISHED |grep 目标IP地址 |wc -l
```

解决：调大net.ipv4.ip_local_port_range='10000 65530'

当作为 client 发请求，或外部流量从 NodePort 进来时进行 SNAT，会从当前 netns 中选择一个端口作为源端口，端口范围由 net.ipv4.ip_local_port_range 这个内核参数决定，如果并发量大，就可能导致源端口耗尽，从而丢包。

net.netfilter.nf_conntrack_tcp_be_liberal

参数值为0或1。

0：表示关闭，所有不在TCP窗口中的RST包都被标志为无效。

1：表示开启，只有不在TCP窗口内的RST包被标志为无效。容器场景下，开启这个参数可以避免NAT过的TCP连接带宽受限。

1.15.11 arm集群中nf_conntrack_tcp_be_liberal参数默认为0

1.17及以上版本nf_conntrack_tcp_be_liberal都已经调整为1

net.core.wmem_max=16777216

net.core.rmem_max=16777216

收发缓冲区内存大小（字节）

netdev-max-backlog溢出丢包

内核从网卡收到数据包后，交由TCP/IP协议栈处理（input_pkt_queue）之前的缓冲队列长度

通过cat /proc/net/softnet_stat的第二列确认

ARP表项垃圾回收 dmesg -T |grep "neighbour: arp_cache: neighbor table overflow"

影响：业务高峰时间段，容器互访频繁，出现ARP表项超过限制，会影响容器之间的互访，出现访问业务超时或DNS解析超时等问题。

原因：ARP缓存超过数量上限（net.ipv4.neigh.default.gc_thresh3默认8192）且无法回收时，新的arp表项无法插入，导致网络包发送时无法找到对应的MAC地址，使得网络包不能发送。

解决：修改集群节点上内核参数net.ipv4.neigh.default.gc_thresh3的值为163790。

vpc网络模式：每个节点从容器网段中划分一个容器小网段，节点最大ARP表项一般不超过为节点个数的平方值

容器隧道模式：容器网络通过隧道封装方式独立于节点网络，共用节点上的主网卡，节点最大ARP表项一般不超过节点个数。

turo集群：每个节点会绑定辅助弹性网卡，节点上的 Pod 共享使用该辅助网卡，每个 Pod 内不会做网络包的路由，只会有一条 ARP 表项，实际的路由控制在节点的 default 命名空间内完成。因此，该网络模式下，ARP 缓存表几乎是共享的，又因为网卡只能属于 1 个子网，因此每个节点的 Pod ARP 缓存表只能存储一个子网的 IP-MAC 映射关系，至多数量为各网卡所在子网内 IP 的数量和，如子网是 /22，即含有约 1000 个 ip，那么 arp 表项也大概有 1000，由于节点网卡配额一般不超过 10，因此该节点的最大 ARP 表项一般不超过 10000。

PID数量、fd数量、内存碎片

timeout: vfork: Resource temporarily unavailable

NetworkManager、systemd-network、dhclient、keepalived等os网络组件

tc qdisc队列

MTU太大

cpu 争抢

单核软中断

软中断线程高一般是网卡收包比较高，如果sar -n DEV 1网卡收包不高，则dmesg看下内核日志，可能是virtio虚拟化层问题

调度延迟

内核bug

unregister_netdevice:waiting for lo to become free

<https://clouddevops.huawei.com/domains/1666/wiki/6/WIKI2022031700857>

ip a：不要grep

是否绑定了eip，是否绑定了nat网关？

如果走nat有问题，先换eip看是否有问题

如果走eip有问题，先换nat看是否有问题

做过安全加固？修改过内核参数？

看下定时任务

echo 3 > /proc/sys/vm/drop_caches

虚拟机规格性能不足，或者有超分（特定规格s6）

虚拟机网卡硬件、驱动

虚拟机网卡开了源/目的检查

源/目的检查功能是I层用来过滤存在非neutron分配的IP和MAC的报文（处于安全考虑，比如OS内随意修改IP和MAC造成冲突的情况），所以这个开关开启后，报文会在虚拟机ply网桥上被过滤掉。

vpc网络模型，跨节点容器间通信场景下，由于容器网段不是neutron分配的，且容器IP未经过封装和snat，经过ecs时会被虚拟机ply网桥认定为非法流量而丢弃

cce vpc网络模型集群创建后默认是关闭的，用户自建k8s 等价于vpc网络模型的话如果打开此开关，会导致跨节点容器间访问不通

容器隧道网络模型下不存在这个问题，容器间访问经过vxlan封装，外层报文的源和目的IP/MAC 都是neutron分配的。

TCS关键操作

看下虚拟机关键操作，有没有热迁移或者其他的操作

三、物理机//虚拟化相关

物理机cpu内存load io高 连接数满 网卡队列堵塞

物理机上其他虚拟机新建连接数，pps突增，影响了其他虚拟机的收发包

虚拟化

四、CCE侧问题

各种资源对象的yaml先看一遍有没有特殊的配置

kube-proxy:

访问不通service ip场景下，看下这个是否正常。

注意模式之间的转换，会导致iptables规则的变化。

注意ipvs模式下大量并发时有一些bug: <https://cloud.tencent.com/developer/article/1826430>

ipvs

注意这个参数net.ipv4.vs.conn_reuse_mode

nginx-ingress参数

超时相关

keepalive相关

canal插件:

节点ping不通pod ip，抓包发现报文没有出虚拟机

查看日志

一般重启canal即可恢复

客户集群升级过程，造成流表丢失No Route to host，master到容器的网络不通。重启canal。

集群内某两个节点之间容器网络不通，canal故障导致的流表丢失

gw接口没有IP，重启下canal

canal网络在节点上抓vxlan报文的方法:

tcpdump -i eth0 'udp and port 4789 and (udp[42:4]=0xAC100016 or udp[46:4]=0xAC100016)' -nve
0xAC100016 为容器ip的16进制表示(AC.10.00.16 -> 172.16.0.22)

udp[42:4]表示源IP，udp[46:4]表示目的IP

yangtse插件，yangtseConfiguration

kubectl get yangtseconfigurations.crd.yangtse.cni default -oyaml

1) 扩节点curl访问容器不通(connection reset by peer)，ping能通，将集群节点网段加入

ipv4PrivateIPAddressRanges中定位思路:

这类问题一般发生在HCSO场景，节点访问容器场景在回程时因识别成集群外节点，导致被SNAT成节点IP，导致connection reset by peer

解决方案: kubectl edit yc修改yangtseConfiguration，将集群节点网段加入ipv4PrivateIPAddressRanges中即可

2) 访问某个公网ip不通 某个网段没有做snat，ipv4PrivateIPAddressRanges配置错误

用户侧自建的网络插件

kubectl get pod -A, 看下有没有安装特殊的网络插件

networkpolicy

容器隧道网络, 四层不通

ingress

参数

证书错误、过期、数量超过限制

service的参数

externalTrafficPolicy: Cluster——> Local Cluster因路由跳转有一点的性能损失

扩展网段限制

有些局点还不支持

service不通

enableServiceLinks: true

这个参数注意下, 会给pod注入环境变量,如果代码中涉及处理env, 则可能造成性能问题

非标准子网网段

切换过node的vpc和子网

service不通

Open vSwitch, ovs-db:

常见于节点开启安全加固导致Open vSwitch与内核版本不匹配

内核不匹配有时候不需要重置节点, 可能重装ovs就好

libgcc权限缩减到只允许root访问。ovs健康检查失败重启, 流表清空No Route to host, 在节点修改/var/pass/script/srvovs-vswitchd中参数, healthcheck代码的ovs-appctl前return 0。kill掉进程ps -ef | grep srvovs, 节点上自动拉起新进程。

五、vpc侧问题

安全组

注意vpc网络模型下要添加容器网段,

注意eni网络模型下pod上关联有安全组

注意安全组优先级, 如果优先级低不生效

注意安全组端口是否连续, 不连续可能失效

ACL

分udp,tcp

注意出规则和入规则都要看

vpc路由表:

VPC网络, 新建或删除节点, 刷新VPC路由表, 时间为节点数*30s, 在路由表刷新完之前, 无法在容器中访问非本节点上的容器。

负载均衡ELB:

黑名单

ELB转发策略存在后端云服务器组下member为空的场景，此时增删后端云服务器，就会导致新下发的配置无法下发

elb的keepalive_timeout默认900s，导致elb连接数被占满

共享：

§ 对于共享型负载均衡，需要确保后端服务器放通100.125.0.0/16网段。

独享

§ 如果开启了“获取客户端IP”，不支持后端服务器和客户端使用同一台服务器，原因是后端服务器会根据报文源IP为本地IP判定该报文为本机发出的报文，无法将应答报文返回给ELB，最终导致回程流量不通。

网段冲突、vpc路由冲突

使用了对等连接、vpn、专线，有多个集群，多个vpc，涉及线下机房等复杂网络情况下，网络冲突、路由冲突的情况很常见

elb, nat, ecs超规格(连接数等)

imissed冲高导致延迟

loop下降会导致延迟

eip带宽超限

常见于偶现超时connect timeout

tsc带宽超限记录工具

工具市场/带宽超速与连接数超限

ddos

安全运维查询

对等连接

多个cce集群或自建k8s之间对等连接场景下，网段冲突导致不通的情况较为常见

CC云连接

打通了云连接，但是没配置域间带宽

DC云专线/物理专线

在pod中通过io.copy上传文件一到晚上就慢，用户侧晚上手动降低了专线带宽

20240312000067 nginx通过容器部署，lb暴露，线下浏览器访问lb地址，下载图片很慢；nginx以虚拟机方式部署仍然很慢；转dc结论：odf架原因

vpn

CFW

20240223000629 CFW侧陈一帆确定客户侧在21号晚上在CFW侧添加一条应用型域名组策略，当前策略仅支持HTTP、HTTPS协议，协议受限，客户侧暂时关闭此策略，测试业务正常。与客户达成一致，于周一输出应用型域名组与地址解析型域名组原理，方便客户进行策略配置

WAF

六、客户端/业务自身问题

客户端侧

只有一家分公司有服务延迟10s，其他分公司正常

分公司客户端配置的证书开启了ocsp特性（Online Certificate Status Protocol），客户端本地校验失败后，使用ocsp访问公网，但是客户是局域网地址，10s超时之后，返回证书不可信

解决：配置根证书和中间人证书的信任后恢复

应用程序自身问题

- 1、查看业务日志，监控；常见原因是java full gc
- 2、请求是否达到服务器端？
- 3、业务程序夯死、死锁，并发bug等等；没开健康检查的话，让用户把http方式的健康检查打开，有时候程序假死日志显示没有收到请求，也没有打印出异常或者错误堆栈看不出来，但是健康检查是可以检查出来这种

业务配置问题

- 1、超时时间配置是30ms，太短了，可能是单位配置错了
- 2、jvm设置（堆内存，新生代老年代等参数）
- 3、问用户要下apm的监控，需要看下业务和jvm相关的监控
- 4、数据库连接池配置？数量太小，超时时间过短？

服务依赖的第三方库bug

常见netty, redisson, lettuce

七、路由器

路由器交换机

物理机上联的交换机的端口上在同时段出现流量突增，流量微突发触发交换机QoS导致出现低概率丢包;由于当时丢包率较低未触发告警，带宽绝对值未达到阈值因此也未触发限制反压

连接云上资源的时候会出现丢失连接的情况。客户目前的资源有ECS、CCE、RDS，都出现了类似的情况，用户修改路由器 TCP MSS 参数后服务正常

八、物理网络

- 100网段防火墙，100网段lvs
- 波分网络 单板故障
- 交换机端口硬件故障，导致 北京四可用区7 东西向链路抖动，导致bgp闪断，路由产生抖动

九、硬件故障

物理机内存异常，影响内存里数据加载顺序，导致网络组件下发内存中的配置异常，影响节点到容器间网络访问。